	AIR UNIVERSITY
	DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING
	EXPERIMENT NO 11

Lab Title: **Huffman Coding**

Student Name: **Muammad Waqas** **Reg. No:** **240756**

Objective: Compression and decompression od data without loss of data using bst and priority , queue

LAB ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Ability to Conduct Experiment					
Ability to assimilate the results					
Effective use of lab equipment and follows the lab safety rules					

Total Marks: _____ Obtained Marks: _____

LAB REPORT ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Data presentation					
Experimental results					
Conclusion					

Total Marks: _____ Obtained Marks: _____

Date: _____ Signature: _____

Department of Computer Engineering

Data Structures And Algorithms

LAB REPORT : 11

Tittle: Huffman-Coding

Name : Muammad waqas

Roll No: 240756

Submitted To : CE . Engr Maryam

Date : 15 December 24

Objective

- **Data Compression:** The primary objective of Huffman coding is to compress data by representing frequently occurring symbols with shorter codes, reducing the overall size of the data.
- **Efficient Encoding:** Huffman coding aims to assign codes to symbols in a way that minimizes the average length of the encoded data, making it more efficient to transmit or store the data.
- **Lossless Compression**: Huffman coding is a lossless compression technique, meaning that the original data can be restored exactly from the compressed data without any loss of information.
- **Variable-Length Codes**: Huffman coding uses variable-length codes, where more frequently occurring symbols are assigned shorter codes and less frequently occurring symbols are assigned longer codes, to achieve better compression ratios.
- **Optimal Prefix Code**: Huffman coding generates an optimal prefix code, which means that no code is a prefix of another code, ensuring that the encoded data can be decoded uniquely and efficientl

====[CODE]====

```cpp
#include<iostream>
#include<string>
#include<queue>
#include<cstring>
using namespace std;
// Node of Huffman Tree
class Node{
public:
    char ch;        // character
    int freq;       // frequency of character
    Node* left;     // left child
    Node* right;    // right child

    // constructor
    Node(char c,int f){
        ch=c;
        freq=f;
        left=right=NULL;
    }
};

// Comparator for priority queue (min heap based on frequency)
class Compare{
public:
    bool operator()(Node* a,Node* b){
        return a->freq>b->freq;   // smaller frequency has higher priority
    }
};
```

====[CODE]====

```
29  class Huffman{
30  private:
31      Node* root;         // root of Huffman tree
32      string code[256];  // store Huffman codes for characters
33
34      // generate Huffman codes using recursion
35      void generate(Node* node,string s){
36          if(node==NULL) return;    // base case
37
38          // if leaf node, store the code
39          if(node->left==NULL && node->right==NULL){
40              code[node->ch]=s;
41              return;
42          }
43
44          // go left with 0
45          generate(node->left,s+"0");
46          // go right with 1
47          generate(node->right,s+"1");
48      }
49
50  public:
51      // constructor
52      Huffman(){
53          root=NULL;
54          for(int i=0;i<256;i++)
55              code[i]="";
56      }
```

====[CODE]====

```
56      // build Huffman tree from frequency array
57      void build(int freq[]){
58          priority_queue<Node*,vector<Node*>,Compare> pq;
59
60          // insert all characters with non-zero frequency
61          for(int i=0;i<256;i++){
62              if(freq[i]>0)
63                  pq.push(new Node((char)i,freq[i]));
64          }
65
66          // build tree by combining two minimum nodes
67          while(pq.size()>1){
68              Node* l=pq.top();
69              pq.pop();
70              Node* r=pq.top();
71              pq.pop();
72
73              // create parent node
74              Node* parent=new Node('\0',l->freq+r->freq);
75              parent->left=l;
76              parent->right=r;
77              pq.push(parent);
78          }
79
80          // final node is root
81          root=pq.top();
82          generate(root,"");   // generate codes
83      }
```

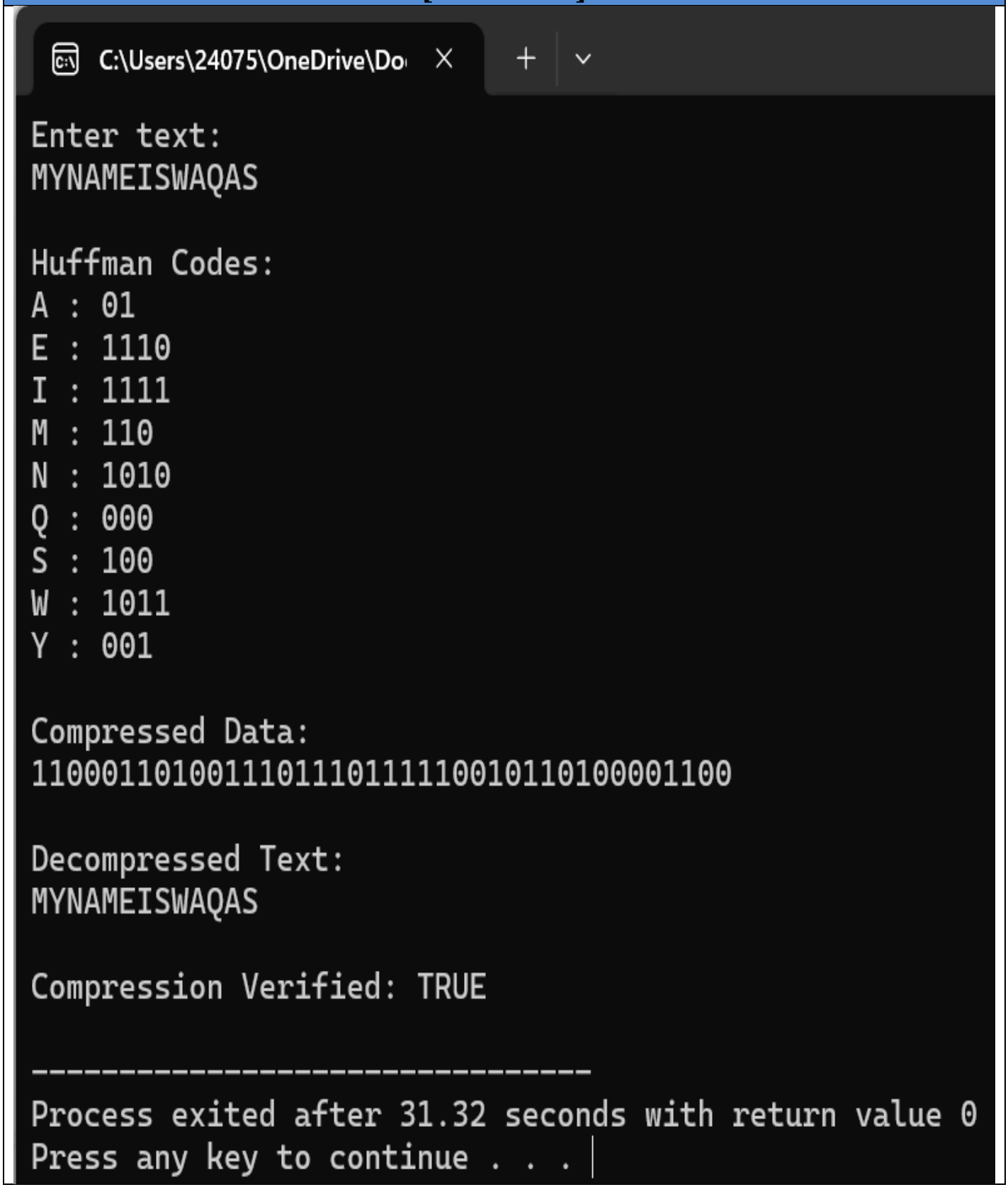
====[CODE]====

```
84      // compress text into binary string
85      string compress(string text){
86          string res="";
87          for(int i=0;i<text.length();i++)
88              res+=code[text[i]];  // replace character with code
89          return res;
90      }
91
92      // decompress binary string back to text
93      string decompress(string bits){
94          string res="";
95          Node* curr=root;
96
97          for(int i=0;i<bits.length();i++){
98              if(bits[i]=='0')
99                  curr=curr->left;
100             else
101                 curr=curr->right;
102
103             // if leaf node reached
104             if(curr->left==NULL && curr->right==NULL){
105                 res+=curr->ch;
106                 curr=root;   // go back to root
107             }
108         }
109         return res;
110     }
```

====[CODE]====

```
111     // print Huffman codes
112     void printCodes(){
113         cout<<"\nHuffman Codes:\n";
114         for(int i=0;i<256;i++){
115             if(code[i]!="")
116                 cout<<(char)i<<" : "<<code[i]<<endl;}}};
117 int main(){
118     string text;
119     int freq[256]={0};   // frequency array
120     cout<<"Enter text:\n";
121     getline(cin,text);
122     // count frequency of each character
123     for(int i=0;i<text.length();i++)
124         freq[text[i]]++;
125     Huffman h;
126     h.build(freq);          // build Huffman tree
127     h.printCodes();         // display codes
128     string compressed=h.compress(text);
129     cout<<"\nCompressed Data:\n"<<compressed<<endl;
130     string decompressed=h.decompress(compressed);
131     cout<<"\nDecompressed Text:\n"<<decompressed<<endl;
132     // verify compression
133     if(text==decompressed)
134         cout<<"\nCompression Verified: TRUE\n";
135     else
136         cout<<"\nCompression Verified: FALSE\n";
137     return 0;}
138
```

====[OUTPUT]====


C:\Users\24075\OneDrive\Do
Enter text:
MYNAMEISWAQAS
Huffman Codes:
A : 01
E : 1110
I : 1111
M : 110
N : 1010
Q : 000
S : 100
W : 1011
Y : 001
Compressed Data:
11000110100111011011111001011010000 1100
Decompressed Text:
MYNAMEISWAQAS
Compression Verified: TRUE
Process exited after 31.32 seconds with return value 0
Press any key to continue . . .

Conclusion:

- Huffman Coding is a **lossless data compression technique**.
- It uses **variable-length binary codes** based on character frequency.
- **More frequent characters get shorter codes**, reducing total size.
- It is built using a **binary tree and min-heap (priority queue)**.
- Original data is **perfectly recovered** after decompression.
- Commonly used in **file compression formats** like ZIP, PNG, and MP3.